

Pilih pernyataan yang **BENAR**.

```
import java.lang.reflect.*;

public class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        Class<?> c = Class.forName("java.lang.String");

        // Baris A
        Object obj = c.newInstance();

        // Baris B
        Method[] methods = c.getMethods();
        System.out.println(methods.length > 0);

        // Baris C: memanggil method 'length' pada String "hello"
        Method lenMethod = c.getMethod("length");
        String s = "hello";
        int len = (int) lenMethod.invoke(s);
        System.out.println(len);

        // Baris D
        Class<?> intClass = int.class;
        System.out.println(intClass.isPrimitive());
    }
}
```

- ☐ A. Reflection tidak dapat digunakan untuk mengakses private field atau private method dalam sebuah kelas.
- ☒ B. Baris B mencetak true karena String memiliki banyak public method yang diwarisi dari Object maupun didefinisikan sendiri.
- ☒ C. Baris C mencetak 5 karena lenMethod.invoke(s) memanggil method length() pada String "hello" yang memiliki 5 karakter.
- ☒ D. Baris D mencetak true karena int.class merepresentasikan tipe primitif int.

Pilih semua pernyataan yang BENAR.

Perhatikan kode berikut:

```
class A { A() { System.out.print("A "); } }  
class B extends A { B() { System.out.print("B "); } }  
class C extends B { C() { System.out.print("C "); } }  
class D extends C { D() { System.out.print("D "); } }
```

Kemudian dieksekusi:

```
D obj = new D();
```



A. Output: "A B C D "



B. Output: "D C B A "



C. Output: "D " saja, karena hanya konstruktor D yang secara eksplisit dipanggil



D. Meskipun tidak ada pemanggilan super() secara eksplisit, Java secara implisit memanggil konstruktor default parent — terjadi constructor chaining sampai ke puncak hierarki

Perhatikan kode berikut:

```
abstract class Bangun {
    abstract double getLuas();

    void cetak() { System.out.println("Luas: " + getLuas()); }
}

class Lingkaran extends Bangun {
    double r;
    Lingkaran(double r) { this.r = r; }

    @Override
    double getLuas() { return Math.PI * r * r; }
}

abstract class BangunBerwarna extends Bangun {
    String warna;
}
```

Kemudian digunakan:

```
Bangun a = new Lingkaran(7);           // Baris 1
a.cetak();                             // Baris 2
BangunBerwarna bb = new BangunBerwarna(); // Baris 3
Lingkaran l = (Lingkaran) a;           // Baris 4
```

- ☐ A. Baris 2 akan mencetak "Luas: 0.0" karena cetak() bukan method milik Lingkaran
- ☒ B. Baris 3 menghasilkan compile error karena BangunBerwarna adalah abstract class dan tidak dapat diinstansiasi
- ☒ C. BangunBerwarna harus tetap dideklarasikan abstract meskipun tidak mendeklarasikan method abstrak baru, karena ia tidak mengimplementasikan getLuas() dari Bangun
- ☐ D. Baris 4 menghasilkan compile error karena casting dari superclass ke subclass tidak diizinkan

Pilih semua pernyataan yang BENAR.

```
public static int divide(int a, int b) {  
    try {  
        System.out.println("try");  
        return a / b;  
    } catch (ArithmeticException e) {  
        System.out.println("catch");  
        return -1;  
    } finally {  
        System.out.println("finally");  
    }  
}  
  
public static void main(String[] args) {  
    System.out.println(divide(10, 2)); // Panggilan X  
    System.out.println(divide(10, 0)); // Panggilan Y  
}
```

- A. Untuk Panggilan X, output yang dihasilkan ☒ adalah: try → 5 (blok finally tidak dieksekusi karena tidak ada exception).
- B. Untuk Panggilan Y, output yang dihasilkan ☒ adalah: try → catch → finally → -1.
- C. Jika return dihapus dari blok catch dan blok finally, method divide() akan tidak bisa dikompilasi karena tidak semua jalur eksekusi mengembalikan nilai. ☐
- D. Blok finally selalu dieksekusi sebelum nilai return dikirim ke caller, baik ada exception maupun tidak. ☐

Pilih semua pernyataan yang BENAR

```
// Method X
static <T> void copy(List<? extends T> src, List<? super T> dst) {
    for (T e : src) dst.add(e);
}

// Method Y
static double sumNumbers(List<? extends Number> list) {
    double total = 0;
    for (Number n : list) total += n.doubleValue();
    return total;
}
```

- ☒ A. Pada Method X, parameter src disebut 'Producer' karena hanya menghasilkan (dibaca) elemen; parameter dst disebut 'Consumer' karena hanya menerima (ditulis) elemen.
- ☐ B. Method Y akan gagal dikompilasi jika dipanggil dengan List<Integer> karena Integer bukan Number.
- ☒ C. Menurut PECS, jika sebuah parameter list hanya akan dibaca maka gunakan <? extends T>; jika hanya akan ditulis maka gunakan <? super T>.
- ☒ D. Pada Method X, baris dst.add(e) akan berhasil dikompilasi bahkan jika dst bertipe List<?> (unbounded wildcard).

Perhatikan kode berikut, **Pilih semua pernyataan yang BENAR.**

```
public class Main {  
    public static void main(String[] args) {  
        String result = 3 + 4 + " hasil " + 3 + 4;  
        System.out.println(result);  
    }  
}
```

☐ A. Output: "34 hasil 34"

☒ B. Output: "7 hasil 34"

☐ C. Output: "7 hasil 7"

D. Operator + dievaluasi dari kiri ke kanan: $3 + 4$ pertama dihitung sebagai aritmetika ($\text{int} + \text{int} =$

☒ 7), sedangkan $3 + 4$ kedua dikonversi ke String karena operand sebelumnya sudah menjadi String

Pilih semua pernyataan yang BENAR

```
List<String> immList = List.of("x", "y", "z");  
Set<Integer> immSet = Set.of(1, 2, 3);  
Map<String,Integer> immMap = Map.of("a", 1, "b", 2);  
  
immList.add("w"); // Baris 1  
List<String> copy = new ArrayList<>(immList); // Baris 2  
copy.add("w"); // Baris 3  
immSet.add(4); // Baris 4  
List<String> nullList = List.of("n", null, "n"); // Baris 5
```

- A. Baris 1 menghasilkan compile error karena
- ☐ List.of() mengembalikan tipe yang tidak memiliki method add().
- B. Baris 1 dan 4 berhasil dikompilasi, tetapi keduanya melempar
- ☐ UnsupportedOperationException saat runtime.
- C. Baris 2 menghasilkan compile error karena
- ☐ tidak bisa membungkus List immutable ke dalam ArrayList.
- D. Baris 3 berhasil dikompilasi dan berhasil dieksekusi karena copy adalah ArrayList yang mutable.
- ☐

Pilih semua pernyataan yang BENAR.

```
public int sqrt(double x) {  
    assert x >= 0 : "x harus non-negatif"; // (1)  
    double result = Math.sqrt(x);  
    assert result >= 0; // (2)  
    return (int) result;  
}  
  
public void setAge(int age) { // public method  
    assert age > 0 : "umur harus positif"; // (3)  
    this.age = age;  
}  
  
// Program dijalankan tanpa flag -ea
```

- ☐ A. Assertion (1) dan (2) tidak berguna karena Math.sqrt() sudah menjamin hasilnya non-negatif.
- ☐ B. Karena program dijalankan tanpa -ea, assertion (1), (2), dan (3) akan melempar AssertionError saat dieksekusi.
- ☒ C. Karena program dijalankan tanpa -ea, assertion (1), (2), dan (3) akan dilewati (tidak diperiksa) oleh JVM.
- ☐ D. Assertion (2) adalah contoh yang baik dari postcondition assertion karena memverifikasi kondisi kebenaran setelah komputasi dilakukan.

Perhatikan desain kelas berikut berdasarkan prinsip SOLID, **Pilih semua pernyataan yang BENAR.**

```
interface Worker {
    void work();
    void eat();
}

class HumanWorker implements Worker {
    public void work() { System.out.println("Working"); }
    public void eat() { System.out.println("Eating"); }
}

class RobotWorker implements Worker {
    public void work() { System.out.println("Working"); }
    public void eat() {
        // Robot tidak makan! Tapi dipaksa implement.
        throw new UnsupportedOperationException();
    }
}
```

- ☐ A. Desain ini sudah baik karena HumanWorker dan RobotWorker sama-sama bisa bekerja
- ☐ B. Desain ini melanggar Interface Segregation Principle karena RobotWorker dipaksa mengimplementasikan method eat() yang tidak relevan
- ☒ C. Solusi yang lebih baik adalah memecah Worker menjadi dua interface terpisah (Workable dan Eatable), sehingga RobotWorker hanya mengimplementasikan Workable
- ☐ D. Desain ini melanggar Liskov Substitution Principle tapi tidak melanggar prinsip SOLID lainnya

Pilih semua pernyataan yang BENAR.

Perhatikan kode berikut:

```
sealed class Shape permits Circle, Square, Triangle { }  
  
final class Circle extends Shape { }  
  
non-sealed class Triangle extends Shape { }  
  
class RightTriangle extends Triangle { }  
  
class Hexagon extends Shape { }
```

- ☐ A. class RightTriangle extends Triangle menghasilkan compile error karena hierarki Shape bersifat sealed
- ☒ B. class Hexagon extends Shape menghasilkan compile error karena Hexagon tidak tercantum dalam permits
- ☒ C. Subclass dari sealed class bebas menentukan apakah ia final, sealed, atau non-sealed; tidak ada kewajiban memilih
- ☒ D. RightTriangle valid karena Triangle dideklarasikan non-sealed, yang membuka kembali hierarki sehingga siapa saja boleh meng-extend Triangle

Pilih semua pernyataan yang BENAR.

Perhatikan kode berikut:

```
class A {  
    protected int x = 10;  
    int getX() { return x; }  
}  
  
class B extends A {  
    int x = 20; // menyembunyikan x milik A  
  
    @Override  
    int getX() { return x; }  
  
    int getSuperX() { return super.x; }  
}
```

Kemudian dieksekusi:

```
A obj = new B();  
System.out.println(obj.x);           // Baris 1  
System.out.println(obj.getX());      // Baris 2  
System.out.println(((B)obj).x);      // Baris 3
```

- ☒ A. Baris 1 mencetak 10 karena akses atribut di
- ☒ Java tidak bersifat polymorphic – ditentukan oleh tipe variabel (A), bukan tipe objek
- ☐ B. Baris 2 mencetak 10 karena getX() milik A yang dipanggil
- ☒ C. Baris 2 mencetak 20 karena getX() bersifat polymorphic dan yang dipanggil adalah B.getX() yang mengembalikan B.x
- ☒ D. Baris 3 mencetak 20 karena setelah di-cast ke B, akses x merujuk ke atribut x milik B

Pilih semua pernyataan yang BENAR.

```
// Versi 1
class PrintThread extends Thread {
    private String msg;
    PrintThread(String msg) { this.msg = msg; }
    @Override public void run() { System.out.println(msg); }
}

// Versi 2
class PrintRunnable implements Runnable {
    private String msg;
    PrintRunnable(String msg) { this.msg = msg; }
    @Override public void run() { System.out.println(msg); }
}

// main()
new PrintThread("A").start();
new Thread(new PrintRunnable("B")).start();
new Thread(() -> System.out.println("C")).start();
```

- ☐ A. Output program dijamin A, B, C secara berurutan karena thread dijalankan secara sekuensial.
- ☒ B. Versi 2 (Runnable) lebih fleksibel dari Versi 1 (extends Thread) karena kelas yang mengimplementasikan Runnable masih bisa meng-extend kelas lain.
- ☒ C. Baris `new Thread(() -> System.out.println("C")).start()` valid karena Runnable adalah functional interface dan bisa diisi dengan lambda expression.
- ☐ D. Memanggil `run()` langsung (tanpa `start()`) tetap menjalankan kode secara multithreaded di thread baru.

Pilih semua pernyataan yang BENAR.

Perhatikan kode berikut berdasarkan prinsip SOLID:

```
class Rectangle {
    protected double width;
    protected double height;

    public void setWidth(double w) { this.width = w; }
    public void setHeight(double h) { this.height = h; }
    public double getArea() { return width * height; }
}

class Square extends Rectangle {
    @Override
    public void setWidth(double w) { this.width = w; this.height = w; }
    @Override
    public void setHeight(double h) { this.width = h; this.height = h; }
}
```

Kemudian digunakan:

```
void testArea(Rectangle r) {
    r.setWidth(5);
    r.setHeight(4);
    assert r.getArea() == 20; // Apakah selalu benar?
}
```

- ☒ A. Jika testArea() menerima objek Square, assertion akan gagal karena getArea() mengembalikan 16, bukan 20
- ☒ B. Desain ini melanggar Liskov Substitution Principle karena Square tidak bisa menggantikan Rectangle tanpa merusak perilaku yang diharapkan
- ☐ C. Masalah ini bisa diselesaikan dengan membuat setWidth() dan setHeight() di Square throw exception
- ☒ D. Contoh ini menunjukkan bahwa relasi "is-a" dalam matematika (Square is a Rectangle) tidak selalu tepat dijadikan hierarki inheritance dalam kode

Pilih semua pernyataan yang BENAR.

Perhatikan kode berikut:

```
class Person {
    String name;
    Person(String name) { this.name = name; }
    void print() { System.out.println("Person: " + name); }
}

class Student extends Person {
    String studentId;
    Student(String name, String studentId) {
        super(name);
        this.studentId = studentId;
    }
    @Override
    void print() {
        super.print();
        System.out.println("ID: " + studentId);
    }
}
```

Kemudian dieksekusi:

```
Person p = new Student("Budi", "13521999");
p.print();
```

- ☒ A. Output pertama: "Person: Budi", output kedua: "ID: 13521999"
- ☐ B. Method print() yang dipanggil adalah milik Student, bukan Person, karena dynamic polymorphism
- ☐ C. super.print() di dalam Student.print() memanggil method print() milik Student sendiri secara rekursif
- ☒ D. Jika Student tidak meng-override print(), maka p.print() akan memanggil print() milik Person dan hanya mencetak "Person: Budi"

Perhatikan hierarki kelas: Object → Animal → Feline → Lion, dan kode berikut, **Pilih semua pernyataan yang BENAR.**

```
List<? super Feline> lf;  
lf = new ArrayList<Animal>();    // Baris 1  
lf = new ArrayList<Feline>();    // Baris 2  
lf = new ArrayList<Lion>();      // Baris 3  
  
Feline f = new Feline();  
Lion l = new Lion();  
Animal a = new Animal();  
  
lf.add(f);                        // Baris 4  
lf.add(l);                        // Baris 5  
lf.add(a);                        // Baris 6  
Object o = lf.get(0); // Baris 7  
Animal x = lf.get(0); // Baris 8
```

- ☐ A. Baris 1, 2, dan 3 semuanya berhasil dikompilasi.
- ☒ B. Baris 1 dan 2 berhasil dikompilasi, tetapi Baris 3 menghasilkan compile error karena Lion adalah subtype dari Feline, bukan supertypenya.
- ☒ C. Baris 6 berhasil dikompilasi karena Animal adalah supertype dari Feline.
- ☐ D. Baris 7 berhasil dikompilasi karena get() pada List<? super Feline> hanya bisa diperlakukan sebagai Object.

Pilih semua pernyataan yang BENAR.

Perhatikan dua desain berikut:

Desain X:

```
class NotificationService {
    private EmailSender emailSender = new EmailSender();
    private SMSGateway smsGateway = new SMSGateway();

    public void notifyByEmail(String msg) {
        emailSender.send(msg);
    }
    public void notifyBySMS(String msg) {
        smsGateway.send(msg);
    }
}
```

Desain Y:

```
interface MessageChannel {
    void send(String msg);
}

class EmailSender implements MessageChannel {
    public void send(String msg) { /* kirim email */ }
}

class SMSGateway implements MessageChannel {
    public void send(String msg) { /* kirim SMS */ }
}

class NotificationService {
    private MessageChannel channel;

    NotificationService(MessageChannel channel) {
        this.channel = channel;
    }
    public void notify(String msg) {
        channel.send(msg);
    }
}
```

- ☒ A. Desain X melanggar Dependency Inversion Principle dan Open/Closed Principle secara bersamaan
- ☐ B. Desain X sudah memenuhi semua prinsip SOLID karena memisahkan email dan SMS ke method yang berbeda
- ☒ C. Desain Y memenuhi DIP karena NotificationService bergantung pada abstraksi (MessageChannel), bukan implementasi konkret
- ☒ D. Desain Y memenuhi OCP karena menambahkan channel baru cukup dengan membuat class baru yang implements MessageChannel, tanpa mengubah NotificationService

17

Perhatikan kode berikut, **Pilih semua pernyataan yang BENAR.**

```
public class Main {  
    public static void main(String[] args) {  
        String a = "Java";  
        String b = a;  
        a = a + "OOP";  
  
        StringBuilder sb1 = new StringBuilder("Java");  
        StringBuilder sb2 = sb1;  
        sb1.append("OOP");  
  
        System.out.println(b);           // Baris 1  
        System.out.println(sb2);        // Baris 2  
        System.out.println(a == b);     // Baris 3  
    }  
}
```

- ☐ A. Baris 1 mencetak "JavaOOP" dan Baris 2 mencetak "Java"
- ☒ B. Baris 1 mencetak "Java" dan Baris 2 mencetak "JavaOOP"
- ☒ C. Baris 3 mencetak true karena a dan b awalnya menunjuk objek yang sama
- ☒ D. Perbedaan output Baris 1 dan Baris 2 disebabkan oleh perbedaan sifat immutable (String) vs mutable (StringBuilder)

18

Pilih semua pernyataan yang BENAR.

Bandingkan kode C++ dan Java berikut:

C++:

```
class Animal {
public:
    void speak() { cout << "..." << endl; }
};
class Cat : public Animal {
public:
    void speak() { cout << "Meow" << endl; }
};
int main() {
    Animal* a = new Cat();
    a->speak(); // Baris X
}
```

Java:

```
class Animal {
    void speak() { System.out.println("..."); }
}
class Cat extends Animal {
    @Override
    void speak() { System.out.println("Meow"); }
}
public class Main {
    public static void main(String[] args) {
        Animal a = new Cat();
        a.speak(); // Baris Y
    }
}
```

- ☒ A. Baris X di C++ mencetak "...", sedangkan Baris Y di Java mencetak "Meow"
- ☐ B. Baris X di C++ mencetak "Meow", sama seperti Baris Y di Java
- ☒ C. Perbedaan ini terjadi karena di C++ method speak() tidak diberi keyword virtual, sehingga pemanggilan di-resolve saat compile-time berdasarkan tipe pointer
- ☐ D. Di Java, harus menambahkan keyword virtual agar polymorphism berfungsi

19

Kasus 1: Kelas

LightningPhoneAsMicroUsbPhone menerima objek LightningPhone di constructor-nya, lalu mengimplementasikan interface MicroUsbPhone dengan cara mendelegasikan pemanggilan ke objek tersebut.

Kasus 2: Kelas StockMarket menyimpan daftar Observer. Setiap kali harga berubah, StockMarket memanggil method update() pada semua Observer yang terdaftar.

Pilih semua pernyataan yang BENAR.

- ☒ A. Kasus 1 adalah contoh Object Adapter pattern karena menggunakan komposisi (menyimpan objek yang diadaptasi) bukan inheritance.
- ☐ B. Pada Kasus 1, jika menggunakan Class Adapter (inheritance), LightningPhoneAsMicroUsbPhone harus mengimplement dua interface sekaligus, yang tidak mungkin di Java.
- ☒ C. Kasus 2 adalah implementasi Observer pattern, di mana StockMarket berperan sebagai Subject (Observable) dan setiap komponen yang mendaftar berperan sebagai Observer.
- ☐ D. Pada Observer pattern, Subject harus mengetahui tipe konkret setiap Observer agar dapat memanggil method yang tepat.

20

Perhatikan kode berikut, **Pilih semua pernyataan yang BENAR.**

```
public class Main {  
    public static void main(String[] args) {  
        String x = "Hello";  
        String y = "Hello";  
        String z = new String("Hello");  
  
        System.out.println(x == y);           // Baris 1  
        System.out.println(x == z);           // Baris 2  
        System.out.println(x.equals(z));       // Baris 3  
  
        String w = "He" + "llo";  
        System.out.println(x == w);           // Baris 4  
  
        String part = "llo";  
        String v = "He" + part;  
        System.out.println(x == v);           // Baris 5  
    }  
}
```

- ☐ A. Baris 1 mencetak true, Baris 2 mencetak false, Baris 3 mencetak true
- ☒ B. Baris 4 mencetak false karena konkatenasi selalu menciptakan objek baru
- ☒ C. Baris 5 mencetak false karena konkatenasi yang melibatkan variabel tidak di-resolve saat compile-time
- ☐ D. Baris 4 dan Baris 5 keduanya mencetak true

Submit